

Contribution Guidelines

Git/GitHub

Commits

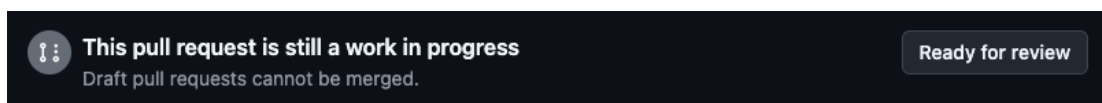
- Follow the [Conventional Commits specification](#) for commit messages.
 - When relevant, scope your commit messages (e.g., `feat(web): implement dbtl component`).
 - Commit messages should be concise yet descriptive.
- Prefer multiple smaller, atomic commits over one large commit. This allows change history to be tracked more easily.

Branches

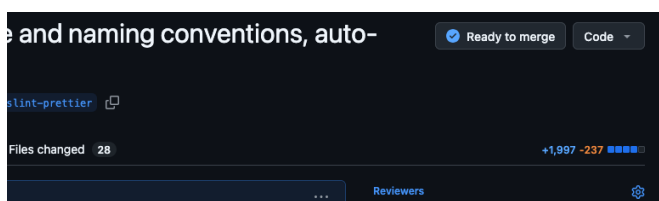
- Always implement features or fixes on a branch.
- Branch names should be descriptive (e.g., `dbtl-component`).
- Do not manually merge a branch into `main` unless advised to do so. Instead, make a PR, which will be merged upon acceptance.
- Do not force push unless you have a good reason to do so (e.g., rebase, fixing broken history). Never force push `main` unless you have a VERY, VERY, VERY good reason and know what you are doing.
- Pull `main` frequently (daily) to ensure your working branch has the latest changes. Either use a [merge commit](#) or a [rebase](#) to integrate changes from `main` . If you have never rebased before, use merge commits. If you are confident in your ability to rebase, it is preferred.

PRs

- Make a **draft** pull request on GitHub as soon as you have a partial implementation.
- The PR should have a short, descriptive title (e.g., `Implement DBTL component`).
- While the PR is not completely implemented, prefix its title with “WIP:” (e.g., `WIP: Implement DBTL component`).
- Leave a brief comment on GitHub listing the overall goals of the PR. Make this a task list for WIP PRs so progress can be monitored.
 - For instance:
 - ☒ Implement DBTL component with support for 4 slots
 - ☐ Implement CSS styling for component
- When your PR is ready for review:
 1. Mark the PR as ready for review.



2. Assign `seb-hyland` as a reviewer (button at top-right of PR page).



- Comments will be left on your PR to request changes if needed.

Style and Code Quality

Tool scripts can be run with `npm run <script-name>`

ESLint

- ESLint has been configured to catch code errors/bad practices and enforce naming conventions.
- You may want to familiarize yourself with the configured rules, especially the requested naming conventions (see [this file](#)).
- Salient conventions to remember:
 - Use `const` over `let` if a variable is not reassigned
 - Use `===` and `!==` over `==` and `!=`
 - Use:
 - `snake_case` for var-like (variable, parameter, property, ...) identifiers
 - `camelCase` for fn-like (function, closure, method, ...) identifiers
 - `pascalCase` for type-like (class, interface, type alias, ...) identifiers
 - `UPPER_CASE` for global identifiers
 - `kebab-case` for file names
- Your editor may provide inline ESLint diagnostics via an extension if properly configured.
- ESLint can be manually invoked via the `lint` script.

Overriding ESLint

- If you are sure your code is correct, you can disable an ESLint error via comments. Here are some common forms:

```
/* eslint-disable <rule-name> */
const a = 0;
/* eslint-enable <rule-name> */

/* eslint-disable-next-line <rule-name> */
const b = 1;

const c = 2; /* eslint-disable-line <rule-name> */
```

Prettier

- Prettier has been configured as a code formatter.
- Formatting rules are enforced by ESLint.
- Prettier extensions are available for a range of editors, often configurable for format-on-save functionality.
- Prettier can be manually invoked via the `fmt` script.

Syntax checking

- Typically, your editor will present Typescript syntax/type errors as diagnostics if it is properly configured.
- Compiler checks can be manually invoked via the `check` script.

Pre-commit

- Before every commit, run the `validate` script.
- This runs linting (incl. format linting) and syntax checking.
- Ideally, the `validate` script should finish without errors on committed project states.
 - If there are warnings/errors associated with unfinished work (e.g., unused variables), make sure you understand the issues.
 - Fix all issues associated with implemented code.

Use of Generative AI Tools

I have no qualms about your use of whatever tool (text completion, LSP completion, Copilot, agents) for writing code. However, I ask the following (which applies equally to handwritten code):

1. Run validation scripts (as discussed above) to catch obvious hallucinations and errors.
2. Read everything. Do not put anything up for review if you have not looked at it.
3. Think through everything. Test, debug, understand what your code does and how it handles various inputs. Do not put anything up for review if you do not understand it.

Hints and principles

This finishing section is *not* a list of hard rules. Instead, it is a series of hints on *how* you might want to write code, and principles to guide you. If you don't like a hint, just throw it away.

Architecture

Guiding principle: Tinkering is trying shit until it works. Engineering is *making based on principle*. As such, software engineering is the *principled* design and implementation of software systems.

1. Before you start programming, understand the problem you are solving. Map out a design for the system at an abstract level, considering what the end-user experience should be.
2. Consider how the system can be discretized into encapsulated components. Consider how these components will communicate: their *interface*.
3. Consider how the abstract behaviour (interface) of each component maps to its implementation.

Syntax and semantics

Guiding principle: Code is a language and *form of media*. As such, it should be written to be read and understood by others — including your future self.

1. Each directory, file, module should be a semantic unit representing a subsystem.
2. Arrange the structure of files to be read. Put the most important contents nearest the top, and less important helpers near the bottom. Group related content together.
3. Consider data and functions as inextricably linked. Data structures are both their data and — just as importantly — their public interface (“methods”).
4. Be descriptive with naming. Prefer `accel` to `a`, `SECONDS_PER_DAY` to `SD`, `validateEmailAddress` to `check`. Worry not about length; modern autocomplete, widescreen displays, and go-to-definition features make description more important than token efficiency. Prefer named arguments when sensible.

```
class PersonBad {  
    private name: string;
```

```

    private id: number;
    private age: number;
    private email: string;

    constructor(name: string, id: number, age: number, email: string) {
        /* Constructor logic */
    }
}
// What do the arguments mean?? Is 57 age or id?
const john = new PersonBad("John Smith", 57, 25, "johnsmith@gmail.com");

class PersonGood {
    private name: string;
    private id: number;
    private age: number;
    private email: string;

    constructor({ name, id, age, email }: { name: string, id: number, age: number, email: string }) {
        /* Constructor logic */
    }
}
// Self-documenting
const jane = new PersonGood({ name: "Jane Doe", id: 25, age: 57, email: "janedoe@icloud.com" });

```

5. Add comments where the intent of code is not obvious to the reader. Restrict comments where the code is obvious and self-documenting.
6. Leverage the type system to mark values with associated semantics. Consider a scenario where we have an API that can download various types of data via an associated ID. By leveraging type-polymorphism hidden behind an interface, we can abstract types behind shared behaviour.

```

class Image { /* definition */ }
class Video { /* definition */ }

interface Downloadable<OutputType> {
    download(): Promise<OutputType>;
}

class ImageId implements Downloadable<Image> { /* definition */ }
class VideoId implements Downloadable<Video> { /* definition */ }

```

7. **Keep it simple, silly.** Try to make the code no more complicated than the system requires. Do not try and create premature generalizations or optimizations; intuition will generally lead you astray in these regards.